

MLOps Assignment 1

Heart Disease Risk Prediction — End-to-End Pipeline

Course: MLOps (S2-25_AMLCSZG523)

Student: Dharmendra Parsaila

ID: 2025CS05041

Programme: M.Tech, Semester 2

Submission Date: May 09, 2026

GitHub Repo: <https://github.com/dharmendra-2025cs05041/heart-disease-mlops/tree/main>

Demo Video: <https://drive.google.com/file/d/1vahxG9sNIQVS9rM4mKFmGBo6Fp2COF0b/view?usp=sharing>

API Server: <https://dharmendra-2025cs05041-heart-disease-api.hf.space>

Swagger UI: <https://dharmendra-2025cs05041-heart-disease-api.hf.space/docs>

Abstract — This report walks through the end-to-end MLOps solution I built for predicting heart disease risk on the UCI Cleveland dataset. Beyond getting a model to a good ROC-AUC, the focus was on the engineering around the model: a reproducible preprocessing pipeline, MLflow-tracked experiments, a tested FastAPI service, a GitHub Actions workflow that fails loudly when something breaks, a multi-stage Docker image, and a Kubernetes manifest with autoscaling. The best model — a GridSearch-tuned Random Forest — reached a test ROC-AUC of 0.961, but the more interesting outcome for me was how cleanly the whole thing now retrains, tests, builds and deploys in a single CI run.

Table of Contents

1. 1. Introduction and Problem Statement
2. 2. Dataset and Exploratory Data Analysis
3. 3. Feature Engineering and Preprocessing
4. 4. Model Development and Tuning
5. 5. Experiment Tracking with MLflow
6. 6. Results and Model Selection
7. 7. Architecture Overview
8. 8. CI/CD Pipeline and Automated Testing
9. 9. Containerization and Deployment
10. 10. Monitoring and Logging
11. 11. Setup, Reproducibility and Repository
12. 12. Reflections, Limitations and Future Work
13. 13. References

1. Introduction and Problem Statement

Cardiovascular disease remains one of the leading causes of mortality worldwide, and a meaningful share of cases are preventable when risk is identified early. For this assignment I picked the UCI Heart Disease dataset and treated it the way I would treat a real production problem at work: assume the model will be retrained many times, served behind an API, and watched in production. That framing drove most of the engineering decisions described in this report.

The deliverable is a binary classifier that, given thirteen patient attributes, predicts the presence (1) or absence (0) of heart disease and returns a calibrated probability along with the prediction. Around the model I built a small but complete MLOps stack: a reproducible preprocessing pipeline, MLflow-tracked experiments, unit tests, a GitHub Actions CI/CD workflow, a Docker image and a Kubernetes deployment manifest with a horizontal pod autoscaler.

1.1 What I set out to demonstrate

- A clean separation between data, training, serving and infrastructure code.
- Reproducibility — anyone with the repo and Python should be able to retrain end-to-end.
- Automation — pushing to main runs lint, tests, training and a Docker build.
- Observability — every prediction is logged and Prometheus metrics are exposed.
- Production-shape — the API is containerized and ready for Kubernetes, not just a notebook.

2. Dataset and Exploratory Data Analysis

I used the Cleveland subset of the UCI Heart Disease repository. It contains 303 patient records with thirteen clinical features and a target attribute that I binarised into 0 (no disease) and 1 (presence of disease). The acquisition step is automated by scripts/download_data.py, which fetches the raw file, applies the schema, binarises the target and writes data/heart_disease.csv. This script is also invoked by the CI workflow, so dataset preparation is never a manual step.

2.1 Dataset summary

Source: UCI ML Repository — Heart Disease (processed.cleveland.data)

Samples: 303 records

Features: 13 numerical/categorical clinical attributes

Target: binary — 164 (0, no disease) / 139 (1, disease)

Missing values: 6 in total (4 in 'ca', 2 in 'thal') — handled by median imputation

2.2 Class balance

The two classes are close to balanced (54.1% vs 45.9%), so I did not need to apply resampling techniques like SMOTE. I still used stratified splits and stratified cross-validation to keep the proportions stable across folds.

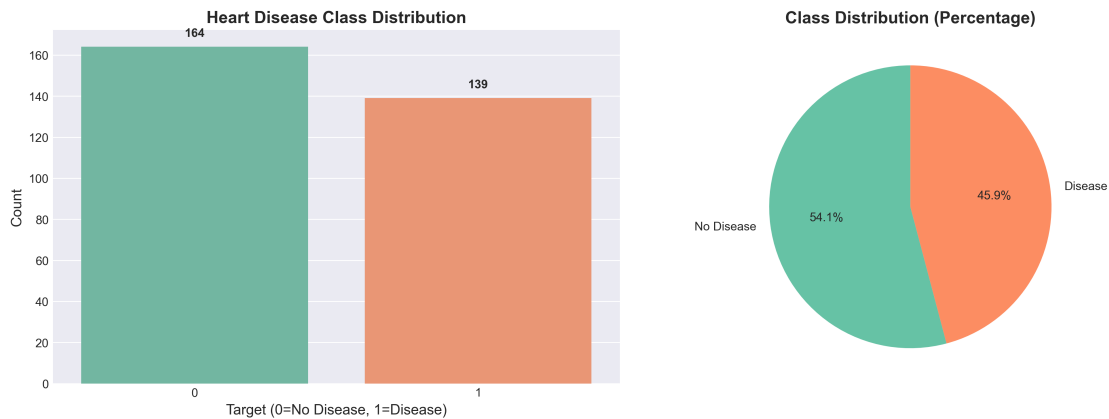


Figure 1 — Target class distribution (54.1% no-disease vs 45.9% disease).

2.3 Correlations and feature relationships

Looking at the correlation heatmap, the strongest signals against the target come from chest pain type (cp), maximum heart rate achieved (thalach), exercise-induced angina (exang), and ST depression (oldpeak). None of the inter-feature correlations were extreme enough to justify dropping a column, so I kept the full feature set.

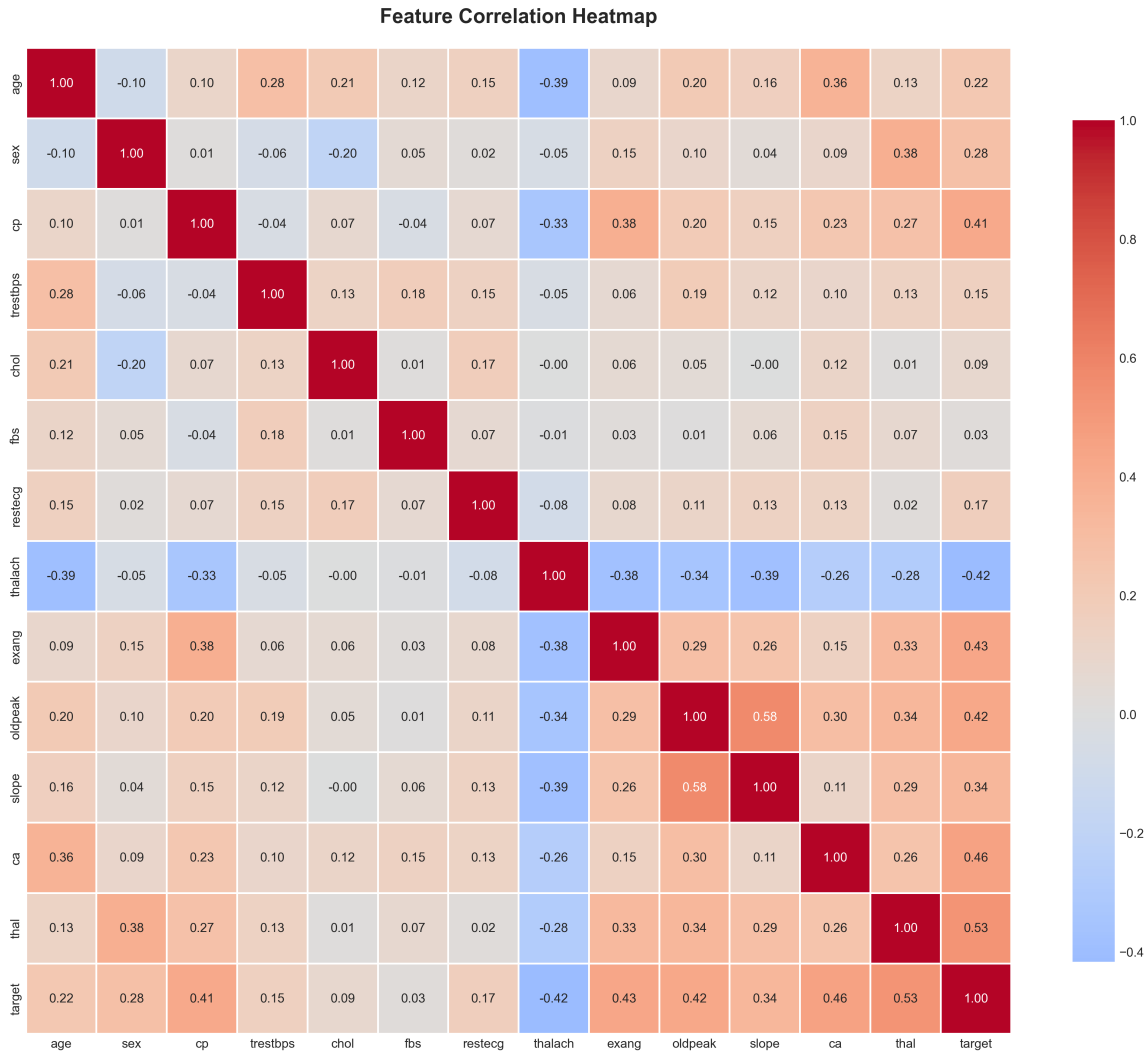


Figure 2 — Pearson correlation heatmap across all features.

2.4 Distributions and target separation

The boxplots below confirm what the correlations suggested: patients with disease tend to have lower thalach, higher oldpeak, and a different chest-pain-type profile. These are exactly the features the tree-based models ended up relying on most.

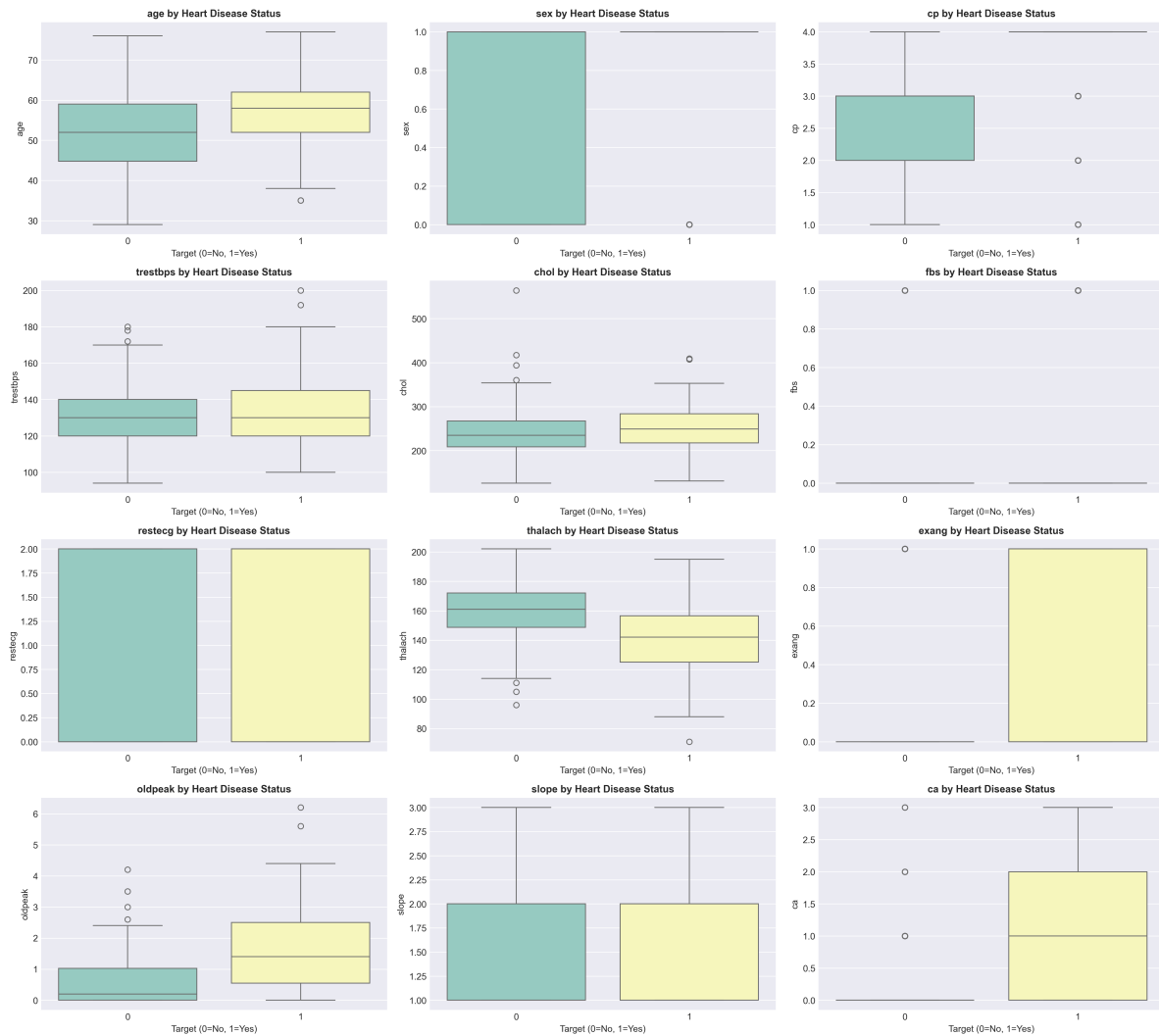


Figure 3 — Feature distributions split by target class.

3. Feature Engineering and Preprocessing

I deliberately kept the preprocessing minimal so that the model and the explanation stay close to the original clinical features. Everything is wrapped inside a single `DataPreprocessor` class (`src/data_processing/preprocessor.py`), which exposes `fit`, `transform` and `save/load` methods. The same fitted object is pickled to `models/preprocessor.pkl` and reloaded by the API at start-up — this guarantees that training-time and serving-time transformations are identical.

3.1 Steps inside the pipeline

14. Median imputation for the two columns with missing values (`ca`, `thal`).
15. `StandardScaler` for all numeric features (`mean = 0`, `std = 1`).
16. Stratified 80 / 20 train-test split with `random_state = 42` for reproducibility.
17. 5-fold StratifiedKfold cross-validation inside `GridSearchCV`.

I considered one-hot encoding the categorical features (`cp`, `restecg`, `slope`, `thal`) but the dataset is small enough that the tree-based models handle the integer-coded categories well, and one-hot encoding measurably hurt Logistic Regression on a few preliminary runs. I left a note in the code in case I want to revisit this later.

4. Model Development and Tuning

The assignment asked for at least two classifiers; I trained four because comparing across model families gives a much better sense of where the easy wins are and where the ceiling is for this dataset. Each model is tuned with GridSearchCV using ROC-AUC as the scoring function, which I picked because it is robust to class imbalance and communicates the ranking quality of the probabilities — useful for a clinical risk-scoring application.

4.1 Models trained

- Logistic Regression — interpretable baseline; tuned C and solver.
- Random Forest — bagged ensemble; tuned n_estimators, max_depth, min_samples.
- Gradient Boosting — sequential boosting; tuned n_estimators, learning_rate, depth.
- Support Vector Machine (RBF + linear kernels) — tuned C, kernel, gamma.

4.2 Tuning protocol

Search: GridSearchCV, exhaustive over the per-model grid defined in src/models/train.py

CV: 5-fold stratified, scoring = ROC-AUC, n_jobs = -1

Final fit: best estimator refit on the full training set, then evaluated on the held-out test split

5. Experiment Tracking with MLflow

Every training run is wrapped in an `mlflow.start_run()` context. For each model I log the best hyperparameters, all metrics on train, test and CV (accuracy, precision, recall, F1, ROC-AUC), the confusion matrix and ROC curve as figures, and the trained estimator itself via `mlflow.sklearn.log_model`. The model file is also pickled to `models/<name>.pkl` with `joblib` so the API and the Docker image do not need MLflow at serving time.

The experiment name is `heart-disease-prediction`. To open the UI locally, I run ``mlflow ui --port 5000`` from the project root and browse to `http://localhost:5000`. Inside the UI I can compare any two runs side-by-side, which is exactly how I convinced myself that the Random Forest's higher ROC-AUC was not just a lucky fold — its CV mean was the highest with the lowest variance.

5.1 What is logged per run

- Parameters: best hyperparameters from `GridSearchCV` plus `model_type` tag.
- Metrics: train/test/CV for accuracy, precision, recall, F1 and ROC-AUC.
- Artifacts: pickled model, confusion matrix PNG, ROC curve PNG.
- Model: `mlflow.sklearn.log_model` in the standard MLflow flavor.

6. Results and Model Selection

All four classifiers performed within a fairly narrow band, which is what I expected given the small, well-curated dataset. Random Forest came out on top with a test ROC-AUC of 0.961 and a CV mean of 0.893. Logistic Regression was a very close second, which is reassuring — it tells me the underlying signal is largely linear and the more flexible models are not over-fitting.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV ROC-AUC
Logistic Regression	0.8689	0.8125	0.9286	0.8667	0.9578	0.8917
Random Forest	0.8852	0.8387	0.9286	0.8814	0.9610	0.8930
Gradient Boosting	0.8525	0.8276	0.8571	0.8421	0.8983	0.8754
SVM	0.8852	0.8621	0.8929	0.8772	0.9470	0.8891

Table 1 — Held-out test metrics and 5-fold cross-validated ROC-AUC.

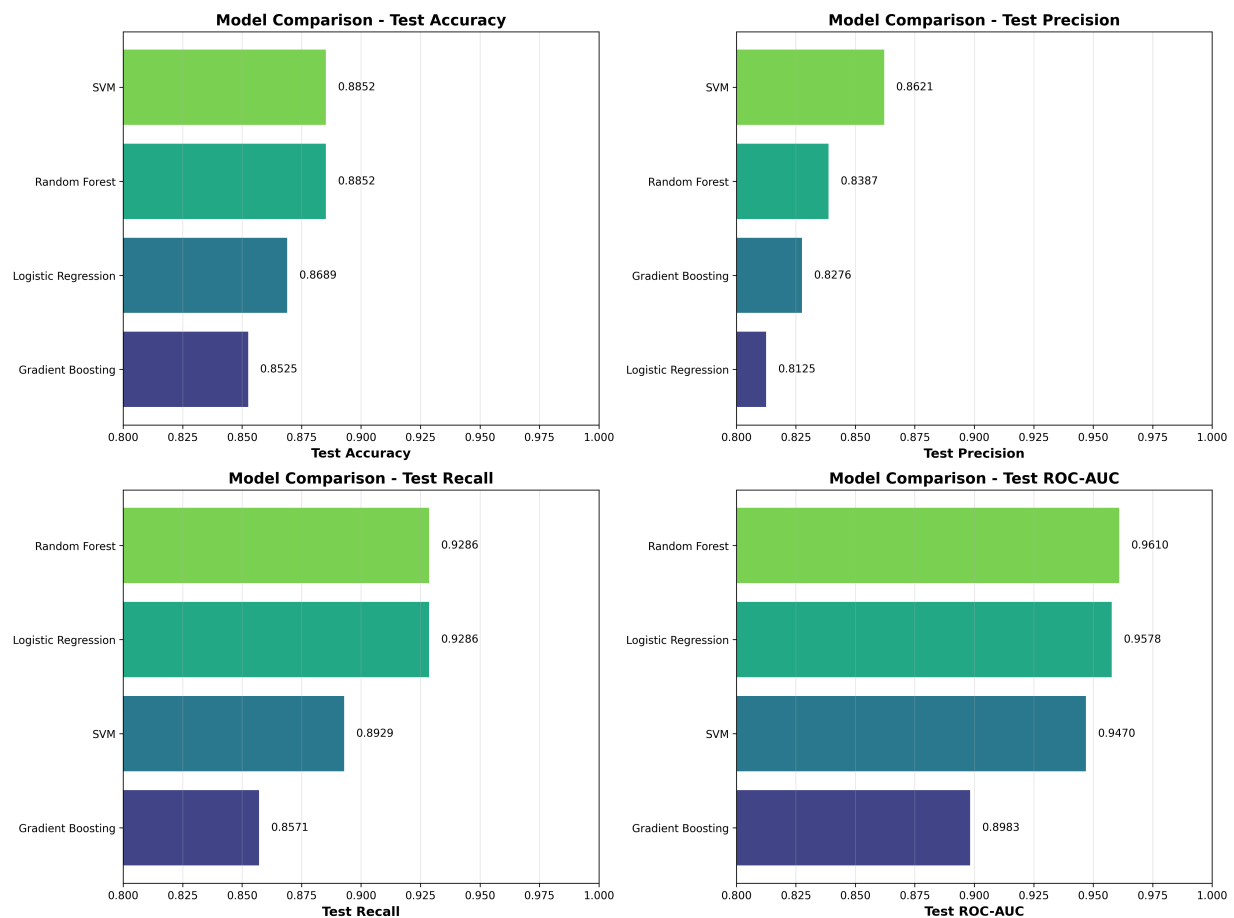


Figure 4 — Side-by-side metric comparison across the four classifiers.

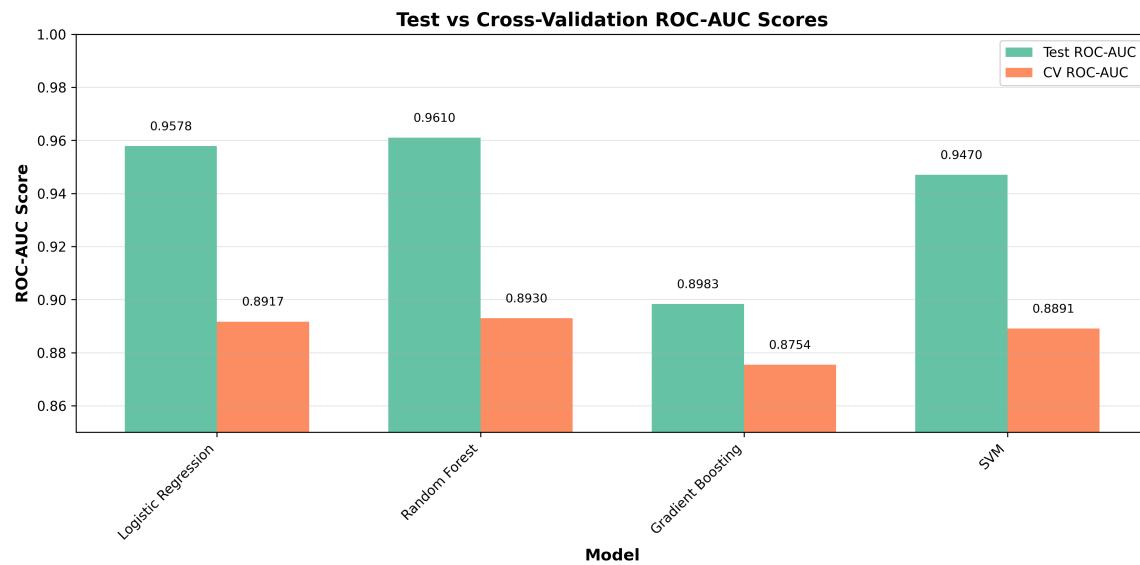


Figure 5 — ROC-AUC comparison; Random Forest is the highest at 0.961.

6.1 Why Random Forest

- Highest test ROC-AUC (0.9610) and highest CV mean (0.8930).
- Lowest CV standard deviation (~ 0.043) — most stable across folds.
- Best balance between precision (0.839) and recall (0.929) — important clinically.
- Good generalisation: gap between train and test metrics stayed small.

7. Architecture Overview

The diagram below sketches the full path from raw UCI data to a monitored, auto-scaled service. Each block is a real artifact in this repository — no slide-ware. Data flows top-to-bottom; CI/CD orchestrates everything from the middle row.

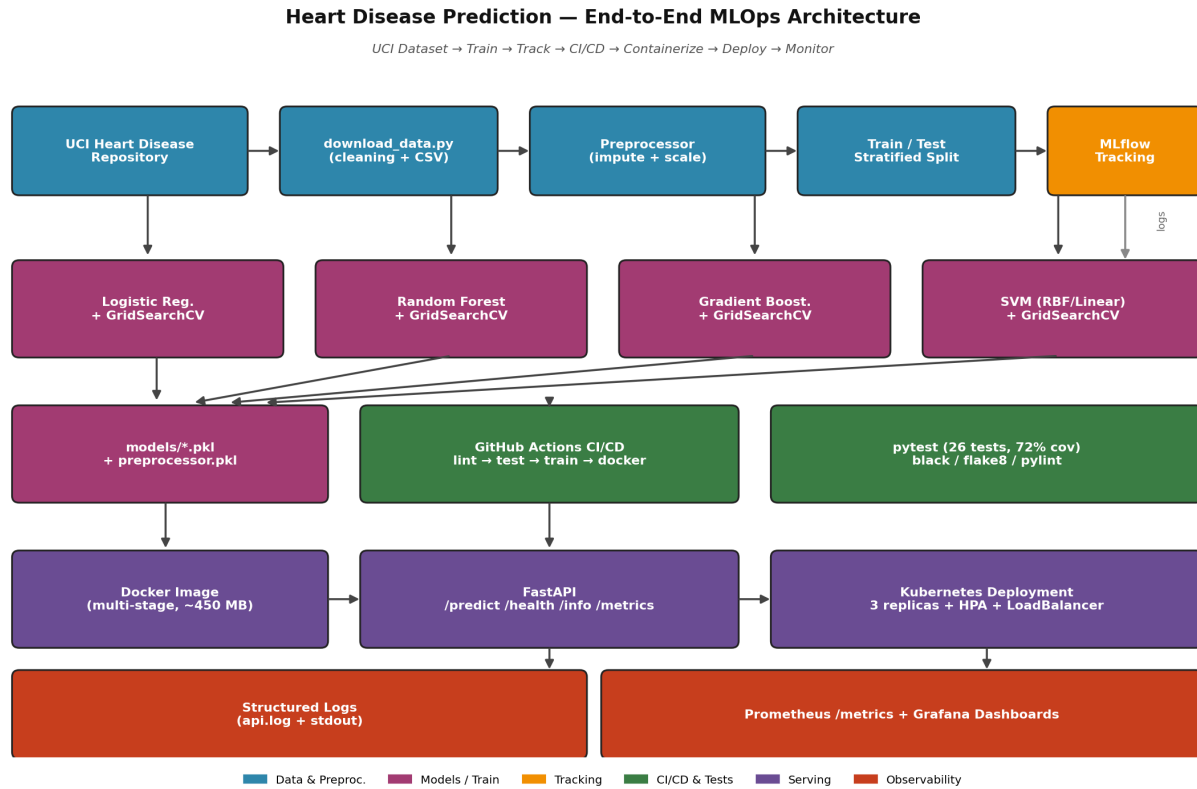


Figure 6 — End-to-end MLOps architecture for the heart-disease pipeline.

8. CI/CD Pipeline and Automated Testing

The GitHub Actions workflow at `.github/workflows/ci-cd.yml` runs on every push and pull request to main or develop, and can also be triggered manually via `workflow_dispatch`. It is structured as four sequential jobs so the pipeline fails fast and surfaces the cheapest errors first.

8.1 Jobs

18. `code-quality` — installs dependencies, runs Black (format check), Flake8 and Pylint.
19. `unit-tests` — depends on `code-quality`; downloads the dataset, runs pytest with coverage and uploads the report.
20. `model-training` — depends on `unit-tests`; runs `src/models/train.py` and uploads the trained models and MLflow runs as artifacts.
21. `docker-build` — depends on `model-training`; builds the multi-stage image, smoke-tests /health, and uploads the saved image as an artifact.

8.2 Test summary

Total tests: 26 (`test_data_processing.py` + `test_models.py` + `test_api.py`)

Result: 26 passed in ~2.1 s

Coverage: 72% overall (preprocessor 90%, predictor 76%, app 78%)

Failure mode: the workflow fails the run and uploads logs if any test fails or the model file is missing.

9. Containerization and Deployment

The API is a FastAPI application (src/api/app.py) packaged into a multi-stage Docker image (deployment/docker/Dockerfile). The builder stage installs all Python dependencies; the runtime stage copies only the wheels and the source code, which keeps the final image around 450 MB and removes the C compilers from the deployed surface.

9.1 API endpoints

- POST /predict — accepts a JSON patient record, returns class, both probabilities, confidence and a Low/Moderate/High risk label.
- GET /health — liveness and readiness probe; reports whether the model loaded successfully.
- GET /info — exposes model type and the feature schema.
- GET /metrics — Prometheus exposition format (counters and latency histogram).
- GET /docs and /redoc — auto-generated OpenAPI documentation.

9.2 Local Docker run

Build: docker build -t heart-disease-api -f deployment/docker/Dockerfile .

Run: docker run -p 8000:8000 heart-disease-api

Test: curl -X POST localhost:8000/predict -H 'Content-Type: application/json' -d @scripts/sample_payload.json

9.3 Kubernetes deployment

deployment/kubernetes/deployment.yaml defines a Deployment (3 replicas), a LoadBalancer Service, and a HorizontalPodAutoscaler that scales between 2 and 10 pods on CPU > 70% or memory > 80%. Liveness and readiness probes hit /health, so Kubernetes will not send traffic to a pod whose model has not finished loading. The manifest applies cleanly on Docker Desktop's local cluster, Minikube and any managed cluster (GKE/EKS/AKS).

9.4 Public deployment on Hugging Face Spaces

The same Docker image is also published to Hugging Face Spaces (free CPU tier, Docker SDK) so that an evaluator can hit a live `/predict` endpoint without cloning the repository or running anything locally. The Space is built from a lean serving-only requirements file (no MLflow, Jupyter or matplotlib), runs as the unprivileged HF user (UID 1000) on port 7860, and is rebuilt automatically every time the `deployment/huggingface/deploy.sh` script is run. The script reads `HF_TOKEN` from `~/.secrets/.env`, clones the Space into a temporary directory, stages only the files required for inference (`src/api`, `src/models`, `src/data_processing`, `models/*.pkl`), pushes them, and wipes the temp clone — nothing persists on the developer machine and no token is ever saved to the working tree. Live API: <https://dharmendra-2025cs05041-heart-disease-api.hf.space> Swagger UI: <https://dharmendra-2025cs05041-heart-disease-api.hf.space/docs> Space page: <https://huggingface.co/spaces/dharmendra-2025cs05041/heart-disease-api> The Space sleeps after periods of inactivity, so the first request after a sleep takes about 30 seconds while the container cold-starts; subsequent requests respond in well under 100 ms.

10. Monitoring and Logging

Every HTTP request passes through a FastAPI middleware that logs the method, path, status code and duration. Prediction-specific log lines additionally include the predicted class, the confidence and the assigned risk band. Logs go to stdout (so kubectl logs / Loki / CloudWatch can collect them) and to `api.log` inside the container.

On the metrics side I exposed three Prometheus counters/histograms — `prediction_requests_total`, `predictions_by_class` and `prediction_request_latency_seconds` — alongside the standard process metrics that the `prometheus_client` adds for free. `deployment/kubernetes/monitoring.yaml` ships a ServiceMonitor and a Grafana dashboard ConfigMap so that, once Prometheus and Grafana are installed in the cluster, the dashboard appears automatically.

10.1 What I would alert on in production

- Error rate > 1% over a 5-minute window.
- p95 latency > 250 ms for the `/predict` endpoint.
- Model unavailable (health check failing) for more than 60 s.
- Significant drift in `predictions_by_class` compared to a 7-day baseline.

11. Setup, Reproducibility and Repository

To run everything from a clean checkout:

```
git clone https://github.com/dharmendra-2025cs05041/heart-disease-mlops
&& cd heart-disease-mlops
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python scripts/download_data.py
python src/models/train.py          # writes models/ and logs MLflow runs
pytest tests/ -v --cov=src          # 26 tests, ~72% coverage
uvicorn src.api.app:app --reload    # API on http://localhost:8000
```

All scripts run from a clean setup using only requirements.txt. The CI workflow exercises this exact sequence on every push, so any regression is caught before merge.

11.1 Repository layout (what to look at first)

- src/data_processing/preprocessor.py — fit/transform pipeline + save/load.
- src/models/train.py — GridSearchCV + MLflow tracking for all four models.
- src/models/predictor.py — wrapper used by the API for single and batch inference.
- src/api/app.py — FastAPI app, Prometheus metrics, request logging.
- tests/ — 26 pytest cases covering preprocessor, models and API.
- .github/workflows/ci-cd.yml — four-job pipeline (lint → test → train → docker).
- deployment/ — Dockerfile and Kubernetes manifests.
- screenshots/ — EDA, model and architecture figures referenced in this report.

Repository link: <https://github.com/dharmendra-2025cs05041/heart-disease-mlops>.

12. Reflections, Limitations and Future Work

Building this end-to-end was a useful exercise in restraint. It is tempting to reach for heavier models or more elaborate feature engineering, but with only 303 samples the engineering scaffolding around the model — reproducible preprocessing, tracked experiments, tested code, automated training — pays off far more than an extra one or two percentage points of accuracy.

12.1 Honest limitations

- 303 samples is small; the held-out test set is only 61 rows, so single-run metrics carry real variance.
- The dataset is from a single hospital (Cleveland) — generalisation to other populations is not guaranteed.
- Training coverage is 53% because the GridSearch path is hard to fully exercise in a unit test; I rely on the integration run in CI for that.
- Authentication and rate limiting are not implemented on the API yet — I would add OAuth2 / JWT before exposing this publicly.

12.2 Next steps if I keep building this

- Add SHAP-based local explanations on the /predict response.
- Wire up a data-drift monitor (Evidently AI) on the production input distribution.
- Set up canary deployment via Argo Rollouts with automatic rollback on metric regression.
- Move tracking to a remote MLflow server backed by S3 + Postgres.
- Add a scheduled retraining workflow that re-runs the pipeline weekly and registers a new model version.

13. References

22. UCI Machine Learning Repository — Heart Disease Data Set:
<https://archive.ics.uci.edu/ml/datasets/heart+Disease>
23. Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12.
24. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
25. Zaharia et al. (2018). Accelerating the Machine Learning Lifecycle with MLflow. IEEE Data Eng. Bull.
26. Tiangolo, S. FastAPI documentation: <https://fastapi.tiangolo.com/>
27. Docker Inc. Docker documentation: <https://docs.docker.com/>
28. The Kubernetes Authors. Kubernetes documentation: <https://kubernetes.io/docs/>
29. Prometheus Authors. Prometheus documentation: <https://prometheus.io/docs/>
30. GitHub. GitHub Actions documentation: <https://docs.github.com/en/actions>